Using AI and Quarto to Transform Higher Education Data Workflows

Workshop handout

Cecile Gleizes and Claire Heacox, Auburn University Office of Institutional Research

October 2026

This handout was written in Quarto and rendered to PDF with Typst. The source is `handout/handout.qmd` in the project. Everything in it applies to the document you are holding.

Getting in

Everything runs on your own laptop. Nothing leaves it. That is a feature, and it is the same arrangement you will want at home with real data.

RStudio Desktop

1. Open the project: double-click `IR-Quarto-Workshop.Rproj`.
2. Files pane (bottom right): `setup/00_install_packages.R`. Click **Source**. Wait for "Done".
3. `setup/01_check_setup.R`. Click **Source**. Ends with "Ready" or tells you what to fix.
4. `exercises/00_hello.qmd`. Click **Render**. A web page opens. You are in.

If step 1 fails because the laptop will not let you install R or RStudio, raise a hand. You will pair with a neighbor for the morning and leave with your institution's report already rendered.

For reference only: Posit also offers RStudio in a browser, Posit Cloud, with nothing to install. We do not use it today. It is a hosted service on servers your institution does not control, with no FERPA agreement unless your institution has made one, so it is not a place for student records.

Before anything else

Your UNITID: _____ (it is in the email; or search `data/institutions.csv` by name)

An AI assistant you can log into: ChatGPT, Claude, Copilot, Gemini. Any.

If you are already in: turn to exercise 1 and start. Do not wait.

The morning

8:30	Getting in	exercise 0
8:45	Why, the landscape, the toolbox	

9:15	One IPEDS question	exercise 1
9:35	Your institution, from the federal data	exercise 2
10:00	Break	
10:10	The room, one click	exercise 3
10:25	Improve it with an AI	exercise 4
10:55	The report you owe someone	exercise 5
11:15	Share, and take it home	

What you will leave with

An IPEDS question answered from a data set in a file that documents itself. A report about your own institution built from the Department of Education's 2026 Program Performance Data, the preview of the earnings accountability results that become official in February 2027. Both improved by an AI you drove. And an outline of the report you actually owe someone, ready to fill.

Why

The problem. More reports, more evidence, more dashboards, more documentation. Same people. The production model has to change, not the hours.

The idea. Put the query, the code, the explanation, the tables, the charts, and the output format in one plain-text file. Render it. Version it. Hand it over. Re-run it next year.

The principle. *The logic is readable; the data never is.* Everything about how a number is made lives in a file anyone can read, including an AI. Student records stay in the system of record and are read at render time.

The landscape, and why RStudio

Where people are	Good at	Weak at
Excel, Access	one-off answers, familiarity	repetition, history, logic hidden in cells
Qlik, Tableau, Power BI	dashboards, drill-down	narrative, PDFs, documentation, version control; licensed per seat
SAS, SPSS, Stata	scripted analysis	the report is still assembled by hand; licensed
Jupyter, Colab, Databricks	Python exploration	publishing; Quarto renders notebooks too
VS Code, Positron	multi-language IDEs	newer; Positron is Posit's next IDE, worth watching

Where people are	Good at	Weak at
RStudio Desktop	free, stable, Quarto built in, one-button render, the IR community	needs an install
Posit Cloud	RStudio in a browser, nothing to install	hosted outside your institution; not for student records; not used today
Posit Workbench and Connect	the server versions: shared, scheduled, published	enterprise licenses

What Auburn built

High school feedback reports. Every year, one Access report printed as a single PDF of more than 3,500 pages, cut by hand into more than 360 files, each named by hand. Now one Quarto template, one parameter, and a loop: 360 PDFs with the right names in one run. The rules were read out of the Access logic and written down before anything was re-implemented, and the old and new were run on the same year and compared figure by figure.

Parent portal retention. A Quarto HTML report for Student Affairs on the office's portal, behind campus sign-in, visible only to the people whose role allows it. The same tool that made the PDFs makes a web page; the portal decides who sees it.

College of Agriculture major changes. Heatmaps on a rolling ten-cohort window that advances by itself each August, with an "about the data" box that states the cohort range and is generated, not typed. A report that updates itself and documents itself.

Faculty salary survey. Twenty-two years in Access: about forty saved queries run by hand in order, a hire date typed into one of them each year, last year's table name typed into another, ten report layouts with hand-edited headers. Underneath, about four rules and thirty-five pieces of scaffolding that existed because Access cannot group and pivot in one query, hold two years in one table, or remember a date. Now four tables and fifteen views, year as a column, nothing updated in place, and one Quarto file that prints all ten reports with the year as a parameter. On the first run it matched the published report to the dollar on 79 of 82 lines.

Department data packets. One PDF per academic department: five years of enrollment, degrees, retention, major changes, faculty, teaching, grades, applications, in the office's style. A "what stands out" box on page two is written by rules from the numbers, so every department gets one and nobody writes sixty of them. One template, one parameter, one loop.

IPEDS. One survey file per component, one section per question, the SQL inside the chunks, checks that print "no errors" or the rows, the upload file written by the last chunk. The Access database, as a document that runs.

Three verbs that are not the same thing

Verb	Means	Where
Run	execute one chunk, or one line, in the Console, right now; the output appears below the chunk; nothing is written to disk	the green arrow on a chunk; Ctrl+Enter on a line
Render	execute every chunk from the top in a fresh session and build the output file (HTML, PDF, Word)	the Render button; `quarto render` in the Terminal
Publish	copy a rendered output somewhere other people can open it	`quarto publish`; or drop the file on a server, SharePoint, the portal

Run is for thinking. Render is for the document. Publish is for the audience. A chunk that Runs but does not Render was depending on something in your session that the document never loaded; Render is the test.

Source and Visual. The two buttons at the top left of the editor. Same file, two views. Source is the plain text: the marks, the fences, the YAML. Visual looks like a word processor. People write in Visual; an AI writes Source, because Source is what you paste into it and what it hands back. Learn to read both. Nothing is lost switching, ever.

What is inside RStudio

More than R. Point at this page during the tour.

Languages, in the same document

Chunk	Runs	Package
`{r}`	R	
`{python}`	Python	`reticulate`
`{sql connection=con}`	SQL against a live connection; result becomes a data frame	`DBI`, `odbc`
`{ojs}`	Observable JavaScript, interactive in the browser	
`{bash}`	shell	

Quarto output formats, one line each

`html` `typst` (PDF, nothing to install) `pdf` (LaTeX) `docx` `pptx` `revealjs` `dashboard` `gfm` (GitHub) and whole websites, books, blogs, manuscripts. Parameters. Conditional content per format. Cross-references. Citations with Zotero. `brand.yml` for institutional colors, fonts and logo in one file.

Packages worth knowing, by job

Job	Packages
Wrangle	`dplyr`, `tidyr`, `janitor`, `stringr`, `lubridate`
Tables	`gt` (HTML, PDF, Word), `kableExtra` (PDF), `flextable` (Word, PowerPoint), `reactable`, `DT` (interactive)
Charts	`ggplot2`, `patchwork`, `ggrepel`, `plotly` (interactive), `leaflet` (maps)
Files	`readxl`, `writexl`, `openxlsx`, `haven` (SAS, SPSS, Stata), `jsonlite`, `httr2` (APIs)
Databases	`DBI`, `odbc`, `dbplyr` (write dplyr, get SQL), `keyring` (passwords out of files)
Office files	`officer` (build Word and PowerPoint from R)
Interactive	`shiny`, Shinylive (Shiny inside a static page)
Workflow	Projects, `renv` (lock versions), the Git tab, Terminal, Background Jobs, `styler`, `esquisse`
Higher ed	`educationdata` (IPEDS by API), `tidycensus`, `rscorecard`, `IPEDSuploadables`
AI	GitHub Copilot (built into RStudio; subscription), `ellmer` (call a model from code), `chattr`

Also exists, not covered today

Shiny makes interactive dashboards: the reader picks a term or a college and the page changes. Same `dplyr` and `ggplot2` inside; it needs a server to run on, which is the cost. Quarto's `dashboard` format and Shinylive give lighter versions with no server. **Posit Cloud** is RStudio in a browser; see the note on the first page.

Publishing

`quarto publish quarto-pub` (free, public). GitHub Pages. Netlify. Posit Connect (licensed). Drop the HTML on SharePoint. Or what Auburn did: an Azure static site behind campus sign-in, with a dashboard that shows each person only the reports they may open.

How IR offices use this

Fact books as websites. Program review profiles, one per department. Course evaluation reports, one per instructor. Common Data Set and IPEDS preparation. Accreditation narratives with the evidence tables embedded. Survey reports from the Qualtrics API. Retention dashboards. Text analysis of comments. Feedback reports to feeder institutions, one PDF each.

Exercise 1: one IPEDS question

Cecile's segment. A data set, a question, and Quarto to get the answer.

1. Open `exercises/01_ipeds.qmd`. Click **Render**.
2. Read the file top to bottom. The data set is a CSV of 3,000 invented students. The question is quoted from the IPEDS form. The rule is one sentence. The chunk filters and counts. The table is the answer.
3. **Try one thing.** Change `attendance == "Full-time"` to `"Part-time"`. Render. You just answered a different line.
4. Copy the whole "The answer" section, paste it below itself, change the filter again. Two questions, two sections. Thirty questions is the same file, longer.

If you know SQL better than R, this is for you. Most of the code in an IPEDS file is SQL. R is the wrapper: it hands the query to the database, catches the result, and shows it or tests it. The last two chunks in the file show it: the same count with the data coming from a warehouse, and the count done in SQL. One chunk, Cecile's shape:

```
qry <- "
  SELECT academic_period, id, program, degree_code, college, dept
    FROM my_schema.degrees_prelim
   WHERE dept IS NULL OR dept = ''
"
error_dept <- dbGetQuery(con, qry)

if (nrow(error_dept) == 0) {
  cat("No department errors.\n")
} else {
  error_dept
}
```

The SQL is the rule. The `if` is the check. The rendered page shows a sentence or the rows. Thirty of these is an error report; thirty counting queries is a survey.

Access to Oracle to Quarto: what changes. Cecile's notes from making the move. The full table is in the primer.

	Access	Oracle	R / Quarto
Crosstab	`TRANSFORM ... PIVOT`	`PIVOT`	`pivot_wider()`
Conditional	`IIf()`	`CASE WHEN`	`if_else()`, `case_when()`
Wildcards	`* ?`	`% _`	pattern in `str_detect()`

	Access	Oracle	R / Quarto
Strings	`"double"`	`'single'`	either
Parameter	`[prompt]`, retyped each run	`:bind`, per query	`params$x`, once per document; a file path can be one
Comments	a hidden properties box	`--` in the SQL	in the code and in the prose around it
One component	30 min to hours, ~30 queries one by one, stalls on big results	5 to 15 min, ~10 queries	up to 5 min, all together, output recorded

A full survey file built this way, with checks that stop the render and an upload file written by the last chunk, is `solutions/ipeds_full_template.qmd`.

Exercise 2: your institution, from the federal data

1. Open `exercises/02_my_institution.qmd`.
2. Near the top, change `unitid: 100858` to your UNITID. The room's are in a comment right there.
3. Click **Render**. Read it. Every number came from `data/`; every table and chart from a code chunk you can see.
4. Change `format:` at the top to `typst`. Render. A PDF, with nothing extra installed. Notice the last section, "Every current program", is gone: in the file it is wrapped in `::: {.content-visible when-format="html"}`. Long tables for the web, short ones for paper, one source. Change it back to `html` when done.

Then look at the file. The YAML block at the top is settings. `params` is the one thing that changes between institutions. Each grey block is a chunk; `#| echo: false` hides the code from the reader. Everything else is Markdown.

What the data is

The **2026 Program Performance Data (PPD:2026)**, published by the Department of Education in December 2025, updated January 8, 2026, for the AHEAD negotiated rulemaking. One row per **institution × credential level × 4-digit CIP**: 209,321 programs at 5,096 institutions. A preview, for information only. The official results under STATS (the Student Tuition and Transparency System, from the One Big Beautiful Bill Act) arrive **February 2027** at the 6-digit CIP level.

Three things to know before computing anything

1. **NA means suppressed, not zero.** Fewer than 10 students (16 for earnings, 30 for FAFSA-based columns) is blanked. Sums undercount.
2. **Counts of 10 to 19 are shown as 15.** Rates built on them use 15.

3. **The dollars are different years.** Earnings and benchmarks: 2024 dollars. Debt: 2019 dollars. Yearly aid: nominal.

The test, in one line

`md_earn_wne_p4` (median earnings of completers working four years after the 2017-18 and 2018-19 cohorts) against `earn_bnchmrk_cip2_wageb` (high school graduates aged 25 to 34 in the state for undergraduate programs; bachelor's holders in the field for graduate programs). Below it: `earnings_test = "Fail"`.

The tables in `data/`

File	One row per	Use for
`ppd_programs.rds`	program	today: earnings, debt, benchmark, pass/fail, demographics
`ppd_yearly.rds`	program × award year	2015-16 to 2024-25: enrollment, completers, Pell, loans
`ppd_cip6_earnings.rds`	program at 6-digit CIP	the earnings detail the official file will use
`institutions.csv`	institution	find a UNITID; compare institutions
`codebook.csv`	column	every column, what it means, where it came from

Exercise 3: the room, one click

1. Open `exercises/03_render_peers.R`. The list is the ten institutions in this room, with UNITIDs.
2. Click **Source**. Ten reports appear in `output/`.
3. Edit the list. Change `format` to `"typst"` if you want PDFs. Source again.

This is the whole trick behind one-PDF-per-college, one-report-per- department, one-profile-per-program. The template does not know how many there are.

If it breaks

Message	Means
`No programs found for unitid`	typo, or your campus reports under a parent OPEID; search `institutions.csv` by name
`could not find function`	a package is not loaded; add `library(...)` in the setup chunk

Message	Means
`object 'x' not found`	a chunk above did not run; Render runs them in order from the top
the PDF table is cut off	shorten column labels or drop a column; or keep it HTML-only

Exercise 4: improve it with an AI

The rule: share the shape of the data, never the rows.

Column names, types, the codebook, your code: yes. Student records: never. Small aggregates can identify people: think first. Passwords and keys: never.
Today's data set is public and has no student-level rows, so you may paste from it. That is the exception. Your warehouse is not like this.

Use whatever assistant you have: ChatGPT, Claude, Copilot, Gemini.

The move

1. Open `02_my_institution.qmd`. Select a chunk, or the whole file. Copy.
2. In the assistant, paste it and ask for **one** change:

   > Here is a chunk from a Quarto report in R: *[paste]*. Change it so that *[one thing]*. Return only the chunk.

3. Paste the result back over the old chunk. Render.
4. If it fails: "Here is the chunk: *[paste]*. Here is the error: *[paste the red text]*. Fix it."

Pick one thing, then another

- Change the chart colors to my institution's colors (give two hex codes).
- Make the margin chart bars sorted the other way.
- Add a column to the test table showing the margin as a percent.
- Add a bar chart of Pell recipients by year under the enrollment chart.
- Add a section at the end with one sentence summarizing the results.
- Make the at-a-glance table two columns side by side instead of a list.

Each one is five minutes. Finished examples of several are in `solutions/improvement_examples.qmd`.

When it answers

Read the code before you paste it. The assistant produces confident code that runs and is wrong. Rendering is the first test; reading the number is the second.

Exercise 5: the report you actually owe someone

Everything so far used our data. This one uses the shape of yours. Think of one report you produce over and over. Not the hardest; the most annoying.

1. Write down its column names and types. Not the data. The names. The primer's "shape of your data" page has the one-liner for wherever it lives: `DESCRIBE` in Oracle, `INFORMATION_SCHEMA` in Snowflake, the Database Documenter in Access, `read_excel(n_max = 0)` for a workbook, or `shape_of()` in `setup/02_shape_of_my_data.R` for anything already

in R. You can also ask the assistant to write the metadata query; that prompt holds nothing sensitive.

2. In the assistant:

> I produce *[report]* for *[audience]* every *[cadence]*. Today I build it in *[Excel / Access / SPSS]* by *[the steps]*. The data has these columns: *[names and types, no rows]*. Outline a Quarto document that produces it, with a parameter for *[the thing that changes each time]*, and write the first two chunks.

3. Make a new file, `my_report.qmd`, in the exercises folder. Paste the outline in. Copy the YAML header from `02_my_institution.qmd` on top.
4. Render it with no data. Headings and empty chunks are a plan you can take home.

At 11:15 we will put a few plans on the screen.

Cheat sheet, and taking it home

The top of a Quarto file

```yaml
---
title: "Report"
date: today
format:
  html:
    toc: true
    embed-resources: true     # one file, images inside
params:
  unitid: 100858
execute:
  echo: false                 # hide code from readers
  warning: false
---
```

In a chunk

```
#| label: name          # unique; shows in Render progress
#| fig-height: 4
#| echo: true           # show this one chunk's code
#| eval: false          # show the code, do not run it
```

Markdown you will use

`# Heading` `## Subheading` `**bold**` `*italic*` `` `code` `` `[text](url)` `1. list` `- bullet` `| a | b |` for tables, `::: {.callout-note}` for a box, `` `r params$unitid` `` for a value inside a sentence.

Render many

```
quarto::quarto_render("report.qmd", execute_params = list(unitid = 100858),
                      output_file = "Auburn.html")
```

Connect to your own warehouse (at home)

```
con <- DBI::dbConnect(odbc::odbc(), dsn = "MY_DSN",
                      UID = "my_user", PWD = keyring::key_get("my_service",
"my_user"))
df  <- DBI::dbGetQuery(con, "SELECT ... FROM ...")
```

`MY_DSN` is an ODBC data source your IT office sets up on your machine. Set the password once in the Console: `keyring::key_set("my_service", "my_user")`. It is stored by the operating system, not in any file, and never in a document like this one.

February 2027

The official STATS file will arrive at 6-digit CIP with its own layout. In this project, `ingest/make_ppd_tables.R` is the one file that reads the Department's files. Change it; every report re-renders.

Links

This project	github.com/AuburnClaire/IR-Quarto-Workshop
PPD:2026 source files	ed.gov, Negotiated Rulemaking 2025-2026, AHEAD Session 2 materials
Quarto	quarto.org/docs/guide
R for Data Science, free	r4ds.hadley.nz
Auburn IR	ir@auburn.edu